Lesson notes · for you

Several agents, one job

AI agents · University · Lesson 2

Time
20 min

Materials
Paper cards and a pen; A printed or hand-copied list of the five checker rules; Your phone, for the lab

Learning goals

- Split an agent into planner, checker, runner and person.
- Write strict rules that stop a bad call before it runs.
- Weigh what more roles cost: tokens and compounding errors.

Activity: Activity: Planner and checker

1. Set-up (3 min): Form groups of three. The planner gets cards and a pen; the checker gets the list of five rules; the runner holds a made-up timetable and a weather list.
2. Play (10 min): Another group writes a request on a card. The planner writes a tool call for it on a new card. The checker tests the call against the five rules and either passes it to the runner or stops it with the rule's name. The runner answers only calls that passed.
3. Swap (5 min): Change roles. The new planner tries to slip a mistake past the checker: a copied name, a 25:00 time, a step after a failure, an action with no yes.
4. Debrief (2 min): Share which slip got through and which rule you would add to catch it.

Write the five rules large enough for the checker to read at a glance. Use only made-up names and classes.

Discussion

- Which mistakes can strict rules never catch?
- When is a second model worth its cost as a checker?
- Why should a stopped call go back to a person instead of a silent retry?

Check yourself

- Write one checker rule as Python or pseudocode, and one slip it would stop.

Try it this week
Pick an agent task you would like to automate, such as booking a study room. Draw its roles and write five strict checker rules, then invent one slip your rules would still miss.

Worksheet

Several agents, one job

AI agents · University · Lesson 2

Name _____ Class _____ Date _____

1 **An agent runs three steps, each right 90% of the time. How often are all three right?**

◯ About 90% ◯ About 73% ◯ About 27%

2 **Write a checker with three rules. check(call, text, on, failed) returns a list of the rules the call breaks: "tool" if its tool isn't in on; "said" if any word in its details doesn't appear in text (the request and the results); "after" if failed is True. An empty list means the call may run. (No model here: the calls are given.)**

```
def check(call, text, on, failed):
    broken = []
    return broken
```

Write your program:

3 **Put the team's steps in order: the planner proposes, the checker reviews, and the call runs only if nothing is broken.**

Number the lines in order (1, 2, 3...). Cross out any line that doesn't belong.

- ☐ `print("Stopped:", broken)`
- ☐ `broken = check(call, text, ON, failed)`
- ☐ `call = planner(request)`
- ☐ `else:`
- ☐ `result = run(call)`
- ☐ `if broken:`
- ☐ `run(call)`

4 **Why start with strict rules in code as the checker, rather than a second language model? Choose all that apply.**

- ☐ Rules never guess, and can't be talked round
- ☐ Rules are cheap and fast at every step
- ☐ Rules catch every possible mistake
- ☐ A model checker can be fooled the same way as the planner
- ☐ Rules understand tone and meaning better than a model

5 **If each of three steps is right 90% of the time, the whole task is right about 90% of the time.**

◯ True ◯ False

✎ **Think of a task where one mistake would really matter, such as messaging a whole class. Which role would you trust a model with, and which would you keep as code or a person?**